

Python Quick Revision Guide

Core Python + Modern Python for AI Development

1. Fundamentals

Topic	Quick Revision
Variables	Python is dynamically typed. Common types: str, int, float, bool, None.
Strings	Immutable. Use strip(), split(), replace(), join(), and f-strings.
Lists	Ordered, mutable. append(), extend(), insert(), remove(), pop(), sort().
Tuples	Ordered, immutable collection; useful for fixed values.
Sets	Unique values; useful for deduplication and membership tests.
Dictionaries	Key-value structure, similar to JS objects. Use .get() for optional keys.
Conditions	if / elif / else; indentation defines blocks.
Loops	for / while; enumerate() gives index + value.
del	del name removes a variable binding; del data['key'] removes a dict key.

2. Structured Python

Topic	Quick Revision
Functions	def function(x: int) -> int:. Keep functions small and focused.
Modules	A .py file containing related code.
Packages	Directory containing related modules; __init__.py is commonly used.
Imports	Prefer clear absolute imports in larger projects.
Exceptions	Use try/except and catch specific exceptions; avoid bare except.
OOP	class, __init__, self, inheritance and composition. Prefer clear responsibilities.
SRP	A function/class/module should have one clear reason to change.

3. Files & Data

Topic	Quick Revision
Files	Use with open(...) as file: so files close automatically.
Paths	Prefer pathlib.Path: Path('data') / 'users.json'.
JSON	load/loads read; dump/dumps write. Use indent=4 for readable JSON.
CSV	Prefer DictReader and DictWriter for named columns.
PDF	Use an appropriate library such as pypdf for PDF extraction/manipulation.
Directories	Path.exists(), is_file(), is_dir(), glob(), rglob().

4. APIs

Topic	Quick Revision
requests	GET/POST/PUT/PATCH/DELETE. Always set timeout.
raise_for_status	Converts 4xx/5xx HTTP responses into exceptions.
Response	status_code, headers, text, content, json().
Content-Type	Validate JSON content type before calling response.json() when needed.
Safe data	Use .get() for optional fields and isinstance() for external type validation.
Errors	Catch Timeout, HTTPError and RequestException as appropriate.
Project layers	Common pattern: routes → services → models/data → utils.
Dependencies	requirements.txt records dependencies; modern projects may use pyproject.toml.

5. Environment & Config

Topic	Quick Revision
.env	Store development configuration/secrets; never commit it.
os.getenv	Reads environment variables; can provide a default.
config.py	Centralize configuration loading.
Secrets	Never hardcode API keys/passwords; use environment variables or a secret manager.
.gitignore	Common: .env, .venv/, __pycache__/, *.pyc.

6. Type Hints

Topic	Quick Revision
Basics	name: str, age: int, price: float, active: bool.
Collections	list[str], dict[str, int], set[str], tuple[int, int].
Optional	Modern Python: str None.
Union	Modern Python: int str.
Returns	def get_name() -> str; use -> None for functions returning nothing.
Important	Type hints do not enforce runtime types by themselves; they help IDEs and static analysis.

7. CLI

Topic	Quick Revision
sys.argv	Raw command-line arguments; argv[0] is normally the script name.
argparse	Preferred for serious CLI tools: help, validation, defaults, choices and type conversion.
Structure	Keep argument parsing separate from business logic.

8. Async

Topic	Quick Revision
I/O-bound	APIs, databases, network/files and cloud storage benefit from async.
CPU-bound	Heavy calculations/model training need CPU resources; async alone does not make them faster.
async def	Defines a coroutine function.
await	Suspends the current coroutine while the event loop runs other work.
asyncio.run	Typical top-level entry point for an async program.
asyncio.sleep	Non-blocking sleep; don't use time.sleep() inside async workflows.
Event loop	Schedules and resumes async tasks, especially while they wait for I/O.

9. Concurrent Execution

Topic	Quick Revision
gather	Run independent awaitables concurrently and collect results; conceptually similar to Promise.all().
create_task	Schedules a coroutine as a Task so it can start while other work continues.
Semaphore	Limits concurrent operations, e.g. asyncio.Semaphore(5).
Timeout	Use asyncio timeout mechanisms to prevent indefinite waits.
Cancellation	Task.cancel() requests cancellation; async code should handle it cleanly.

10. Production Checklist

Topic	Quick Revision
Quality	Readable names, small functions, single responsibility, type hints and focused modules.
Reliability	Timeouts, specific exceptions, retries/backoff where appropriate, input/output validation.

Topic	Quick Revision
Security	No secrets in source control; validate external input; use least privilege.
Observability	Use logging; track latency, errors and external API usage.
Testing	Keep business logic testable; cover important edge cases.
AI readiness	These foundations support FastAPI, Pydantic, OpenAI SDKs, RAG, MCP, LangGraph and Agentic AI.